Classic and Modern Fraud Detectors under Controlled Imbalance, Drift, Ring Fraud and Label Delay: A Reproducible Study on Synthetic Data

Thanatchon Chongmankhong

October 2026

Abstract We compare classic and modern fraud detectors on synthetic transaction data whose fraud mechanisms, timing, label latency and relational structure are known and controllable. A seeded generator with eight pluggable fraud scenarios produces about 200,000 transactions per dataset, and a leakage-tested harness enforces strict temporal splits and past-only features. Five research questions were pre-registered with numeric refutation criteria, and every comparison uses five paired seeds, Holm correction and a 0.02 AUPRC equivalence margin. Tuned LightGBM reached a test AUPRC of 0.738 at 0.5% fraud prevalence, against 0.488 for FT-Transformer, and no neural model (FT-Transformer, a GRU sequence model or a time-respecting GraphSAGE) beat the tuned tree ensemble in any comparison where a difference could be resolved; TabPFN could not be evaluated because its weights require a license token. LightGBM with hand-crafted graph features was never beaten by GraphSAGE and beat it clearly when ring fraud was rare, while graph features improved LightGBM once at least half of the fraud was ring-based. Operating conditions mattered more than the choice of model family. A mid-stream switch in fraud tactics cut every supervised model's AUPRC by 88% or more; weekly refitting recovered only part of the loss within two weeks, and retraining with a 10-day validation holdout recovered none. A 7–30-day label delay cost tuned LightGBM 53% of its AUPRC, and adding unsupervised anomaly scores did not help. Model rankings by AUPRC and by recovered loss per investigator-hour agreed only partially (Kendall $\tau = 0.73$). All results are synthetic and are reproduced by a single command.

1 Introduction

Transaction fraud detection is a well-established application of machine learning, yet its empirical literature is hard to build on. Most strong claims rest on proprietary data from a bank or platform [1], [2], [3], [4], and the one widely used public card dataset is anonymised by PCA, which leaves no account, device or merchant structure to study. Three practical questions are therefore poorly answered. Does the recent wave of tabular deep learning and tabular foundation models help in the extreme-imbalance regime where fraud lives [5], [6]? Do graph neural networks earn their complexity over gradient-boosted trees given good relational features [7], [8]? And how do the operational facts of fraud, namely adversarial drift, label delay and limited investigator capacity [1], [9], change the ranking of model families?

We answer these questions on **synthetic data whose fraud mechanisms are known and controllable**. Our contributions are:

1. **`fraudlab.synth`**, a seeded, parameterised generator with eight pluggable fraud scenarios (card testing, account takeover, velocity abuse, friendly fraud, bust-out, mule networks, merchant collusion and synthetic identities), drift schedules, a label-delay model, and a typed account–merchant–device graph. The output is content-hashed and bitwise reproducible.

2. **Five pre-registered research questions** whose hypotheses, refutation criteria and numeric thresholds were fixed before any data was generated (repository file `docs/RESEARCH_QUESTIONS.md`).
3. **A leakage-tested evaluation harness**: strict temporal splits, past-only features verified by future-truncation tests, a time-respecting GraphSAGE sampler, five paired seeds per comparison with Holm correction, and an equivalence margin.
4. **Negative results stated plainly.** The modern tabular transformer did not beat a tuned gradient-boosted tree at any fraud rate. GraphSAGE did not beat a tree ensemble given hand-crafted graph features. Adding anomaly scores did not offset label delay. Validation-holdout retraining did not recover from drift at all.

Everything in this paper, including every number in the text and tables, is regenerated by `make reproduce` on a fanless laptop in about 48 minutes.

2 Related work

Classical learners and practitioner evaluation. Random forests and gradient boosting on engineered aggregation features are the long-standing industrial baseline [10], [11]. Dal Pozzolo et al. formalised the operating conditions of a fraud-detection system: an alert budget, investigator feedback and delayed labels. They argued for top-k alert precision over AUC [1], and showed that undersampling distorts probabilities in a correctable way [12]. Cost-sensitive learning frames the decision as thresholding calibrated probabilities [13], [14]. Independent tabular benchmarks consistently find tree ensembles hard to beat [15], [16], but these benchmarks deliberately exclude severe imbalance and temporal splits.

Unsupervised and semi-supervised detection. Isolation Forest [17], one-class methods [18] and autoencoders [19] don't need labels. ADBench reports that even about 1% labelled anomalies beat the best unsupervised detector [20]. Adding outlier scores as features to a supervised model gives modest gains on real card data [21].

Sequence models. LSTMs and GRUs over a card's history complement engineered features [22], and feature-free interleaved RNNs are reported to beat a production tree system [2]. Transformers over transaction sequences continue this line [23], [24]. All of these results use proprietary data or unequal feature sets.

Graph methods. GNNs designed for fraud handle camouflage [25] and class imbalance [26], and heterogeneous transaction graphs have been deployed at scale [3], [27]. On Elliptic, however, a random forest on node features beat a GCN [7], and GADBench finds tree ensembles with neighbourhood aggregation competitive with or better than specialised GNNs [8]. No study varies how much relational structure the fraud actually has.

Foundation-model era. FT-Transformer is the strongest of the simple deep tabular baselines [5], and SAINT adds row attention and pretraining [28]. TabPFN performs in-context prediction on up to 10,000 rows [6], [29]. At Booking.com, transformers pretrained on large volumes of traffic beat tuned GBDTs under selection bias [4]. LLMs can engineer features [30] but lag specialised classifiers when used directly on tabular fraud data [31], [32].

Synthetic fraud data. PaySim simulates mobile money with one main fraud pattern [33]. AMLworld generates laundering typologies with complete labels [34]. BAF provides GAN-generated account-opening data with bias variants [35], and the ULB handbook ships a simple three-scenario simulator with a delay-aware protocol [36]. Generic tabular generators [37] need real data to fit. Our generator combines pluggable multi-scenario fraud, drift schedules, label delay and a device graph in one seeded library.

3 Synthetic data framework

World model. Accounts have heterogeneous activity (Gamma-distributed rates), log-normal amounts with account- and category-level locations, von Mises time-of-day preferences, five habitual merchants (65% of card payments) and three P2P contacts. Card-present payments carry no customer device, while online payments and transfers do. A single `noise` knob scales legitimate look-alikes of fraud signals: trips abroad, devices shared within a household, phone switches, one-off devices and unusually large purchases. It was set to 0.4 by a pre-registered difficulty check that looked only at validation data (repository `docs/DECISIONS.md`, D014).

Fraud scenarios. Each scenario is a class that emits the transactions of one fraud event. The generator repeats events until a per-segment budget (fraud rate × legitimate volume) is met, and trims the time-ordered tail of the last event so realised rates are exact. Three scenarios are *ring* scenarios defined by relational structure: mule networks (victims ⟶ young mule accounts on shared devices ⟶ a collector), colluding merchants (a new merchant billing stolen cards) and synthetic identities (fresh accounts sharing devices). Friendly fraud is the cardholder's own normal-looking purchase, later disputed. It is close to undetectable by design and caps achievable recall.

Labels. Each fraud event is detected with probability 0.95. Its label arrives after a delay drawn from Uniform(7, 30) days, plus jitter, and 1.5× longer for friendly fraud. Undetected fraud stays labelled legitimate forever.

Presets. The main experiments use `small`: 197,300 transactions, 20,101 accounts, 1,006 merchants and 21,585 devices over 30 days, at a 0.5% fraud rate for seed 0. The time-dependent RQs use `small_long`: 199,519 transactions over 90 days, the same row budget with a longer timeline. Every dataset is regenerated from its seed inside the experiment, and its content hash is logged with each run.

4 Experimental setup

Features. Each transaction gets 24 past-only *tabular* features: amount, category, channel, time of day, account and merchant age, velocity counts and sums over 1 h, 24 h and 7 d, amount relative to the account's 30-day mean, device novelty, transfers received, and known prior fraud on the account. It also gets 11 *graph* features: accounts per device, counterparty fan-in and sender count, merchant customer base, known-fraud counts on the device, merchant and counterparty, and the account's connected-component size and PageRank in a start-of-day snapshot. A feature for a transaction at time T uses only earlier transactions and labels available before $\min(T, \text{deployment})$. A test checks that features are unchanged when all data after an arbitrary time is deleted.

Models. Logistic regression (C chosen on validation), random forest (300 trees), LightGBM with library defaults, and LightGBM tuned by Optuna TPE on validation AUPRC with early stopping (30 trials in RQ1, 15 in RQ3/RQ4, equal across compared arms). Isolation Forest and an MLP autoencoder are unsupervised references. The GRU encodes an account's previous 16 transactions together with the current row's features. FT-Transformer [5] uses $d = 64$, 3 blocks and 8 heads. GraphSAGE uses two mean-aggregator layers over a time-respecting, uniformly sampled neighbourhood (fan-out 10, 5) of the transaction's account, merchant or counterparty, and device. The three neural models train on all positives plus a fresh 10% negative sample per epoch, up to 20 epochs with early stopping on validation AUPRC. Their probabilities are corrected for undersampling [12]. TabPFN [6] is implemented (CPU, a stratified temporal subsample of at most 10,000 training rows), but its weights require a license token. It was skipped in the reported run (`results/skipped.json`: package not installed (uv sync –extra tabpfn)).

Protocol. Splits are strictly temporal: 60/15/25% of the timeline for train, validation and test. The model is frozen at the start of the test window ("deployment"). Unless stated otherwise, training and validation labels are those of detected fraud ("oracle"), and the evaluation target is ground truth, including undetected fraud. Each seed regenerates the dataset and re-seeds every model, and all models share

the dataset within a seed, so comparisons are paired. The primary metric is test AUPRC. Secondary metrics are ROC-AUC, daily precision@k and recall@k ($k = 20$ alerts/day), F1 at the validation-optimal threshold, ECE, and latency.

Statistics. For each pre-registered comparison we report the mean paired difference over 5 seeds, its 95% t-interval, and a Holm-adjusted paired t-test within each RQ. "A beats B" requires adjusted $p < 0.05$ *and* a mean difference of at least $\delta = 0.02$ AUPRC. "Equivalent" requires the 95% CI to lie within $\pm\delta$. Everything else is *inconclusive*.

5 Results

All results are on test windows that no model or tuning procedure saw. Tables and figures are generated from the run logs in `results/runs/`.

5.1 RQ1: Old vs. new under extreme imbalance

Table 1: Test AUPRC (mean ± std over 5 seeds) by fraud rate, `small` preset.

Model	AUPRC @ 0.1% fraud	AUPRC @ 0.5% fraud	AUPRC @ 1.0% fraud
LightGBM (tuned)	0.318 ± 0.258	0.738 ± 0.066	0.795 ± 0.014
LightGBM (default)	0.048 ± 0.079	0.620 ± 0.106	0.739 ± 0.015
Random forest	0.576 ± 0.177	0.726 ± 0.046	0.784 ± 0.022
Logistic regression	0.215 ± 0.153	0.326 ± 0.068	0.423 ± 0.075
GRU	0.202 ± 0.054	0.552 ± 0.091	0.673 ± 0.058
FT-Transformer	0.036 ± 0.036	0.488 ± 0.071	0.656 ± 0.060
Isolation Forest	0.153 ± 0.117	0.189 ± 0.079	0.195 ± 0.060
Autoencoder	0.035 ± 0.035	0.032 ± 0.011	0.046 ± 0.014

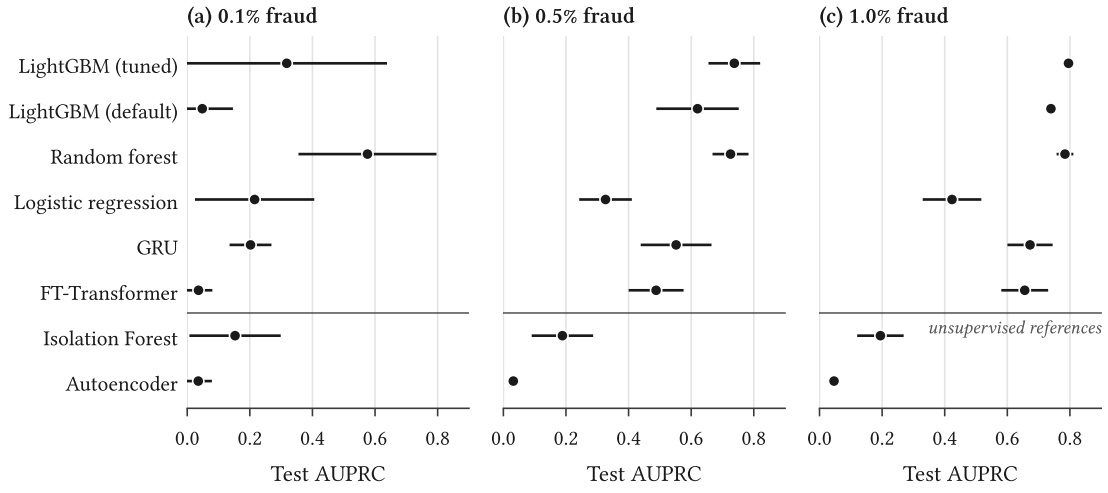


Figure 1: RQ1: test AUPRC by fraud rate on the `small` preset, mean and 95% confidence interval over 5 paired seeds. Isolation Forest and the autoencoder, below the rule, are the unsupervised references.

At 0.5% and 1% fraud, tuned LightGBM had the highest mean AUPRC (0.738 ± 0.066 and 0.795 ± 0.014), with random forest close behind. FT-Transformer reached 0.488 ± 0.071 and 0.656 ± 0.060. The paired differences against tuned LightGBM were −0.250 (95% CI −0.364 to −0.135) and −0.140 (-0.211 to −0.069). At 0.1% fraud the gap was −0.282, with a Holm-adjusted p of 0.063. The modern model never beat the tuned GBDT, so **H1a is not refuted**. Note that this holds for FT-Transformer only. TabPFN, the second pre-registered modern arm, did not run.

The 0.1% condition is the noisiest. Only a few dozen frauds fall in each test window, and every model's interval is wide. In this condition random forest had the highest mean (0.576 ± 0.177) and tuned LightGBM reached 0.318 ± 0.258. This comparison was not pre-registered and is reported as exploratory only. Default LightGBM degenerated at 0.1% (0.048 ± 0.079). With no L2 regularisation and very few training positives, its predictions saturated at 0 and 1. Tuning improved on default by +0.270 (95% CI +0.039 to +0.501), but the Holm-adjusted p was 0.063. The pre-registered rule counts an inconclusive result as a refutation, so **H1b is refuted**, although the point estimate favours tuning.

Table 2: RQ1 pre-registered comparisons (paired over seeds, Holm-adjusted within RQ1).

Fraud rate	A	B	Mean Δ AUPRC (A–B)	95% CI	Holm p	Verdict
0.1%	FT-Transformer	LightGBM (tuned)	−0.282	[−0.564, -0.000]	0.063	inconclusive
0.5%	FT-Transformer	LightGBM (tuned)	−0.250	[−0.364, -0.135]	0.015	LightGBM (tuned) beats FT-Transformer
1.0%	FT-Transformer	LightGBM (tuned)	−0.140	[−0.211, -0.069]	0.016	LightGBM (tuned) beats FT-Transformer
0.1%	LightGBM (tuned)	LightGBM (default)	+0.270	[+0.039, +0.501]	0.063	inconclusive

Table 3: Secondary metrics at 0.5% fraud. Latency is per 1,000 predictions on the test window, and fit time includes tuning. Isolation Forest and the autoencoder produce scores rather than probabilities, so no ECE is reported for them.

Model (0.5% fraud)	ROC-AUC	Precision@k	Recall@k	F1 @ valid thr.	ECE	ms / 1k preds	Fit s
LightGBM (tuned)	0.983 ± 0.005	0.750 ± 0.138	0.487 ± 0.062	0.710 ± 0.070	0.005 ± 0.009	2.2 ± 1.3	74.0 ± 49.0
LightGBM (default)	0.968 ± 0.010	0.680 ± 0.139	0.439 ± 0.052	0.620 ± 0.052	0.002 ± 0.000	0.5 ± 0.0	0.5 ± 0.0
Random forest	0.975 ± 0.007	0.733 ± 0.141	0.476 ± 0.072	0.714 ± 0.051	0.007 ± 0.004	2.4 ± 0.1	8.9 ± 0.4
Logistic regression	0.948 ± 0.013	0.399 ± 0.096	0.258 ± 0.047	0.360 ± 0.053	0.002 ± 0.001	1.3 ± 0.0	0.8 ± 0.0
GRU	0.963 ± 0.007	0.606 ± 0.175	0.390 ± 0.077	0.549 ± 0.092	0.002 ± 0.001	4.8 ± 0.2	6.5 ± 0.4
FT-Transformer	0.965 ± 0.009	0.528 ± 0.055	0.348 ± 0.065	0.522 ± 0.050	0.002 ± 0.001	30.5 ± 0.3	38.1 ± 0.6
Isolation Forest	0.867 ± 0.031	0.296 ± 0.083	0.189 ± 0.027	0.171 ± 0.063	–	8.5 ± 0.2	0.6 ± 0.0
Autoencoder	0.795 ± 0.034	0.031 ± 0.042	0.023 ± 0.031	0.054 ± 0.015	–	1.3 ± 0.0	0.6 ± 0.0

Cost matters as well. Tuned LightGBM needed 74 s to tune and fit, mostly for its 30 Optuna trials, against 38 s for FT-Transformer. FT-Transformer inference took 30.5 ms per 1,000 predictions, against 2.2 ms for LightGBM.

5.2 RQ2: Fraudster adaptation (concept drift)

Table 4: Static models deployed on day 50. Degradation is $D = 1 - \text{AUPRC}_{post}/\text{AUPRC}_{pre}$. The pre-drift window holds 4 days of test data.

Model (static)	Type	AUPRC pre-drift (d50–54)	AUPRC post-drift (d54–90)	Degradation D	95% CI of D
Logistic regression	supervised	0.713 ± 0.280	0.042 ± 0.025	0.94 ± 0.03	[0.90, 0.97]
LightGBM (tuned)	supervised	0.909 ± 0.112	0.107 ± 0.046	0.88 ± 0.06	[0.81, 0.95]
GRU	supervised	0.875 ± 0.134	0.067 ± 0.040	0.92 ± 0.04	[0.87, 0.98]
Isolation Forest	unsupervised	0.197 ± 0.211	0.207 ± 0.021	−2.05 ± 3.46	[−6.34, 2.24]
Autoencoder	unsupervised	0.031 ± 0.015	0.028 ± 0.005	−0.56 ± 1.90	[−2.91, 1.80]

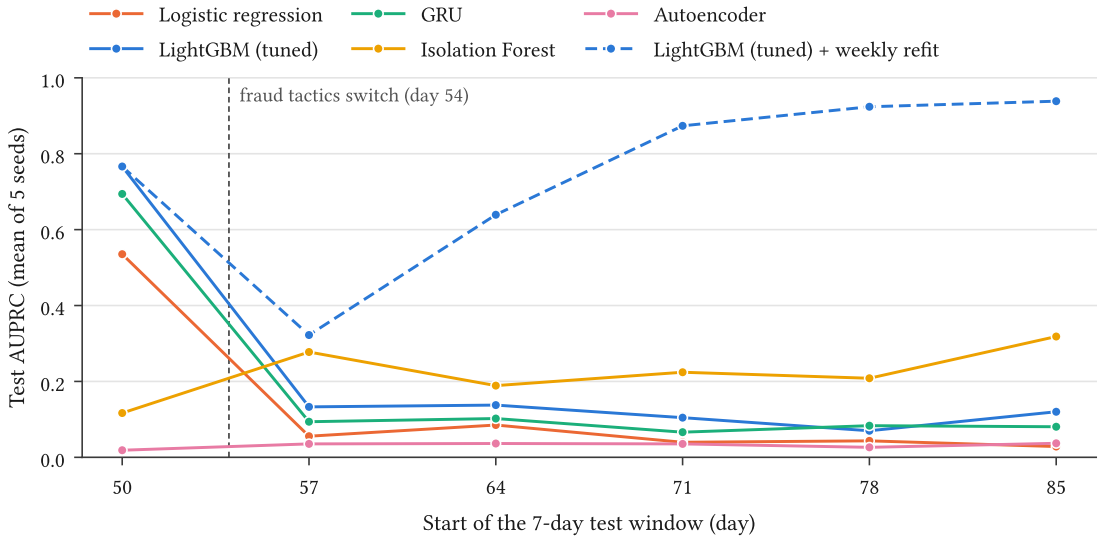


Figure 2: RQ2: weekly test AUPRC before and after the drift on the `small_long` preset, mean of 5 seeds. The fraud tactics switch at day 54. Solid lines are models frozen at day 50; the dashed line is LightGBM refit weekly on all labels available at refit time.

Once fraud switched from card-level attacks to bust-out, synthetic identities and mule rings, every supervised model collapsed. Logistic regression lost 94% of its pre-drift AUPRC, LightGBM 88% and the GRU 92%, with all intervals well above the 30% threshold. The unsupervised detectors did not degrade on average, but only because they started near the floor: Isolation Forest scored 0.207 after the drift, against 0.107 for static LightGBM. **H2a is not refuted.** After the drift, the weakest model before it, Isolation Forest, was the best static model, but none was useful.

Table 5: Recovery over days 61–68, $R = (\text{AUPRC}_{policy} - \text{AUPRC}_{static})/(\text{AUPRC}_{pre} - \text{AUPRC}_{static})$.

Quantity (LightGBM, days 61–68)	Mean ± std
AUPRC pre-drift (static, d50–54)	0.909 ± 0.112
AUPRC static	0.152 ± 0.099
AUPRC weekly refit on all labels (H2b)	0.346 ± 0.241
Recovery R, refit	0.26 ± 0.23 (95% CI [−0.02, 0.54])
AUPRC weekly re-tune with 10-day holdout	0.103 ± 0.078
Recovery R, holdout re-tune	−0.07 ± 0.05 (95% CI [−0.14, -0.00])

Refitting LightGBM weekly on all labels available at refit time (the pre-registered policy) gave a recovery of $R = 0.258 \pm 0.226$ (95% CI [−0.023, 0.539]). The interval reaches just past the 0.5 threshold, so **H2b is not refuted**, but the point estimate is about half the pre-registered target. By days 71–85 the refit model was back near its pre-drift level (Figure 2). The operational variant, which re-tunes on data that excludes a 10-day validation holdout, recovered nothing (R = -0.069 ± 0.055). Its training data simply ended before the new fraud began. *A validation holdout delays adaptation by its own length.* This variant was added after the first run exposed the issue (repository DECISIONS D029).

5.3 RQ3: Graph models vs. tabular models with graph features

Table 6: Test AUPRC and ring-scenario recall@k (k = 20/day) by the share of fraud coming from ring scenarios.

Ring share	AUPRC: LightGBM (tuned)	AUPRC: LightGBM + graph features	AUPRC: GraphSAGE	Ring recall@k: LightGBM (tuned)	Ring recall@k: LightGBM + graph features	Ring recall@k: GraphSAGE
0%	0.718 ± 0.037	0.714 ± 0.054	0.421 ± 0.097	–	–	–
25%	0.702 ± 0.076	0.718 ± 0.061	0.423 ± 0.101	0.543 ± 0.205	0.742 ± 0.152	0.210 ± 0.159
50%	0.735 ± 0.072	0.798 ± 0.048	0.453 ± 0.197	0.718 ± 0.105	0.778 ± 0.097	0.556 ± 0.257
75%	0.791 ± 0.102	0.842 ± 0.110	0.590 ± 0.235	0.772 ± 0.085	0.771 ± 0.120	0.605 ± 0.251

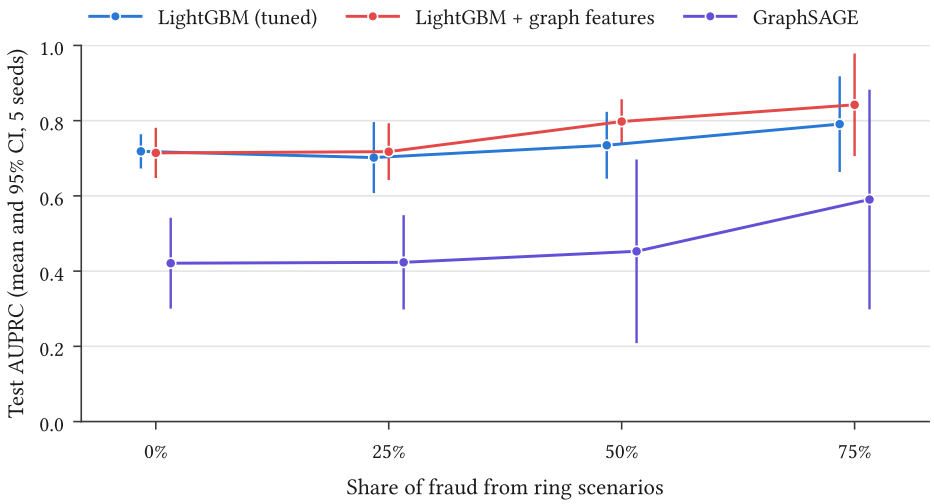


Figure 3: RQ3: test AUPRC as the share of fraud from ring scenarios increases (`small` preset, 0.5% fraud), mean and 95% confidence interval over 5 seeds. Points are offset slightly within each share so that the intervals do not overlap.

GraphSAGE was well below both LightGBM variants at every ring share. At 50% ring share its AUPRC was 0.453 ± 0.197, against 0.798 ± 0.048 for LightGBM with graph features. No comparison favoured GraphSAGE, so **H3a is not refuted**. Per-share verdicts are in the RQ3 test table below. At the higher ring shares GraphSAGE's seed-to-seed variance is large, and those comparisons are borderline or inconclusive rather than decisive. Graph features did help the tree model once rings were common, by +0.063 AUPRC at 50% and +0.051 at 75%. **H3b is not refuted.** With no ring fraud, the graph features made no difference (0.714 with them vs. 0.718 without). Relational signal therefore matters, but in this setting a GNN is not needed to use it. This agrees with GADBench [8] and with the Elliptic baselines [7]. Our GraphSAGE was not tuned and uses a simple node featurisation, which is a threat to this conclusion (Section 6).

Table 7: RQ3 pre-registered comparisons.

Hyp.	Ring share	A	B	Mean Δ AUPRC	95% CI	Holm p	Verdict
H3a	0%	GraphSAGE	LightGBM + graph features	−0.293	[−0.404, −0.183]	0.007	LightGBM + graph features beats GraphSAGE
H3a	25%	GraphSAGE	LightGBM + graph features	−0.294	[−0.350, −0.239]	0.001	LightGBM + graph features beats GraphSAGE
H3a	50%	GraphSAGE	LightGBM + graph features	−0.345	[−0.620, −0.070]	0.051	inconclusive
H3a	75%	GraphSAGE	LightGBM + graph features	−0.252	[−0.572, +0.069]	0.095	inconclusive
H3b	50%	LightGBM + graph features	LightGBM (tuned)	+0.063	[+0.027, +0.099]	0.025	LightGBM + graph features beats LightGBM (tuned)
H3b	75%	LightGBM + graph features	LightGBM (tuned)	+0.051	[+0.037, +0.066]	0.003	LightGBM + graph features beats LightGBM (tuned)

5.4 RQ4: Label delay

Table 8: Test AUPRC (days 60–90) for models trained at day 60 with every detected fraud labelled (oracle), or only those whose label had arrived (delayed; unlabelled fraud counts as legitimate).

Model	AUPRC oracle labels	AUPRC delayed labels	Relative loss	95% CI of loss
Logistic regression	0.338 ± 0.087	0.280 ± 0.050	0.15 ± 0.15	[−0.03, 0.33]
LightGBM (tuned)	0.698 ± 0.068	0.340 ± 0.204	0.53 ± 0.25	[0.22, 0.84]
GRU	0.544 ± 0.063	0.123 ± 0.126	0.78 ± 0.21	[0.51, 1.04]
LightGBM + anomaly scores	0.683 ± 0.079	0.167 ± 0.126	0.77 ± 0.15	[0.58, 0.96]

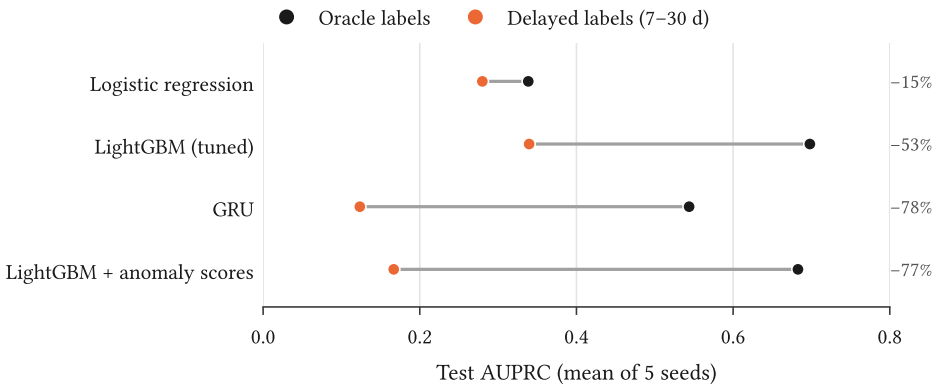


Figure 4: RQ4: test AUPRC with oracle labels and with delayed labels on the `small_long` preset, mean of 5 seeds. The figure at the right of each row is the relative loss, $1 - \mathrm{AUPRC}_{delayed}/\mathrm{AUPRC}_{oracle}$.

Delay hurt every supervised model. Tuned LightGBM lost 53% of its AUPRC, the GRU 78%, and logistic regression, which had the least to lose, 15%. No model's interval lay below 10%, so **H4a is not refuted**. Adding Isolation Forest and autoencoder scores did not mitigate the loss. The recovery estimate was −0.734 ± 0.869 (95% CI [−1.813, 0.344]), meaning the augmented model did *worse* than plain LightGBM under delay. **H4b is formally not refuted, but only because the interval is extremely wide.** The evidence points against the hypothesis.

Table 9: Training labels visible and the recovery of the semi-supervised variant.

Quantity	Value
Training positives visible (oracle)	534 ± 93
Training positives visible (delayed)	445 ± 84
AUPRC lost by LightGBM (oracle − delayed)	0.358 ± 0.144
Recovery by LightGBM + anomaly scores	−0.73 ± 0.87 (95% CI [−1.81, 0.34], seeds with a loss: 5)

The size of these losses has a second cause, found after the run (DECISIONS D030). Under delay, the validation window has almost no matured labels: 4 of 5 seeds had zero visible validation frauds, against 52–148 with oracle labels. Delayed models were therefore tuned and early-stopped essentially blind. RQ4 measures the combined cost of fewer training labels and of losing the validation signal, which is how a naive pipeline experiences delay.

5.5 RQ5: Cost and investigator capacity

Table 10: Operational metrics at 0.5% fraud with 20 alerts per day (3.3 investigator-hours per day).

Model (0.5% fraud)	AUPRC	Recovered $ / investigator-hour (rank by p)	… (rank by p × amount)	Total cost $ (missed + 6.67/alert)
LightGBM (tuned)	0.738 ± 0.066	2507 ± 1178	3167 ± 1440	42688 ± 18342
LightGBM (default)	0.620 ± 0.106	2022 ± 1202	2987 ± 1649	55627 ± 20243
Random forest	0.726 ± 0.046	2498 ± 1343	3141 ± 1597	42943 ± 15645
Logistic regression	0.326 ± 0.068	1161 ± 446	1885 ± 809	78595 ± 35720
GRU	0.552 ± 0.091	1101 ± 544	2100 ± 762	80178 ± 36308
FT-Transformer	0.488 ± 0.071	1384 ± 393	2281 ± 952	72644 ± 36157
Isolation Forest	0.189 ± 0.079	1408 ± 776	n/a (score)	71996 ± 23629
Autoencoder	0.032 ± 0.011	57 ± 86	n/a (score)	108019 ± 45092

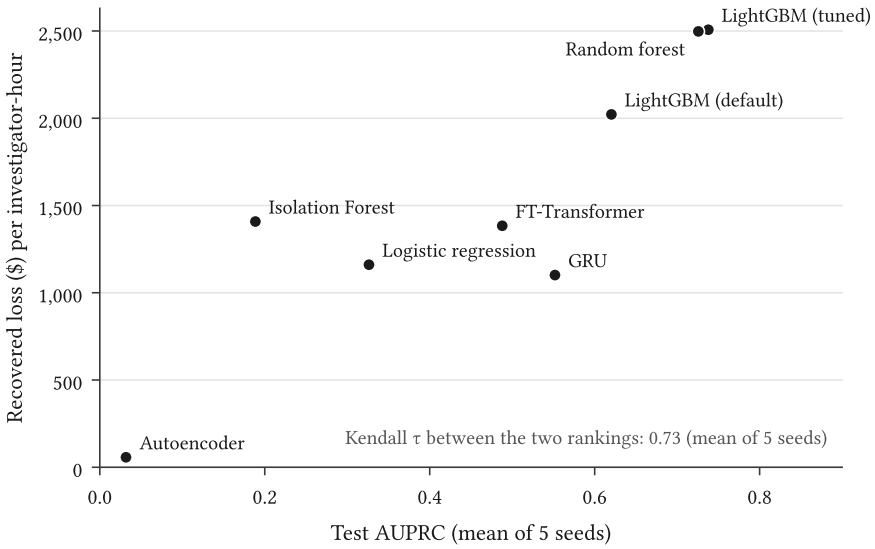


Figure 5: RQ5: ranking quality against operational value at 20 alerts per day (`small` preset, 0.5% fraud): test AUPRC against recovered loss per investigator-hour, mean of 5 seeds.

With capacity fixed, tuned LightGBM recovered the most loss per investigator-hour ($2,507), and random forest was effectively tied. Within the alert budget, tuned LightGBM caught 61% of the test-window fraud losses, against 29% for logistic regression. Ranking alerts by expected loss ($p \times$ amount) instead of p raised recovered loss per hour for the best model by 27% (95% CI 9% to 46%), so **H5a is not refuted**. The lower bound sits just under the 10% target. The mean Kendall τ between model rankings by AUPRC and by recovered loss per hour was 0.73, below the pre-registered 0.8, so **H5b is refuted**. Isolation Forest is the clearest example. It ranks near the bottom on AUPRC ($1,408 per hour) but recovers about as much per hour as FT-Transformer ($1,384) and more than the GRU ($1,101).

6 Threats to validity

Synthetic is not real. This is the principal threat. Our scenarios are clean mechanisms with stylised parameters. Real fraud mixes tactics, adapts to the deployed model, and lives in far richer feature spaces. Absolute AUPRC values are not comparable with any published real-data result, and we make no claim that a model ranking here transfers to a given institution. What does transfer is the *conditional* evidence: under mechanism X, model family Y behaves like Z.

Construct validity. Friendly fraud is near-undetectable by construction, which caps every model's recall. The ring scenarios produce few events per dataset (DATA_CARD limitation 3), so RQ3 has low power. Label delay is modelled as independent of model decisions, so there is no investigator feedback loop (FUTURE_WORK).

Internal validity. Hyperparameter budgets are modest and unequal across families. LightGBM gets Optuna; the neural models get fixed, literature-default architectures with early stopping. A larger neural budget might change RQ1. Thresholds were pre-registered, but some protocol choices were made after early infrastructure runs and are documented with their reasons, including one in which two test values were visible (DECISIONS D024–D028).

Statistical conclusion validity. Five seeds give wide intervals, and the three-way verdict deliberately reports "inconclusive" rather than over-reading them.

7 Reproducibility statement

All code, configurations and pre-registered hypotheses are public under Apache-2.0. `make reproduce` regenerates every dataset from its seed, reruns all experiments, writes `results/summary.json`, and

renders this paper from a template in which every number is a placeholder filled from that file. Dependencies are pinned in `uv.lock`, and generated datasets carry a SHA-256 content hash. The headline run used arm64 hardware (arm, 10 cores, 16 GB, PyTorch 2.14.1, MPS available: True), as reported by `fraudlab probe` (`results/hardware.json`). The suite took 48 minutes of wall-clock time in two parallel lanes. Neural results may differ in the last decimal across MPS and CPU, while generator output is bitwise identical across runs.

8 Conclusion

On controlled synthetic fraud data, the classic toolkit held up. A tuned gradient-boosted tree, given good past-only features and, where relevant, hand-crafted graph features, matched or beat a tabular transformer, a sequence model and a graph neural network in every pre-registered comparison where a difference could be resolved. The modern models did not help here, and in some regimes they cost more compute for less accuracy. This is not a claim that they never help: TabPFN could not be evaluated, the neural models received fixed rather than searched architectures, and the data are synthetic. The operational questions mattered more than the choice of model family. A change in fraud tactics wiped out 88% or more of every supervised model's ranking quality, a realistic label delay removed 53% of the best model's, and the speed of recovery depended more on the retraining pipeline (refit on everything vs. hold out a validation window) than on the model. For practitioners this suggests three priorities: invest in retraining cadence and label latency, rank alerts by expected loss rather than by probability, and judge models on recovered value under capacity, because AUPRC ranks them only approximately. The generator, harness and every number here are reproducible with `make reproduce`, and we invite extensions through new scenarios and models.

9 References

[1] A. Dal Pozzolo, G. Boracchi, O. Caelen, C. Alippi, and G. Bontempi, "Credit card fraud detection: A realistic modeling and a novel learning strategy," *IEEE Transactions on Neural Networks and Learning Systems*, 2018, doi: 10.1109/TNNLS.2017.2736643.

[2] B. Branco, P. Abreu, A. S. Gomes, M. S. C. Almeida, J. T. Ascensão, *et al.*, "Interleaved sequence RNNs for fraud detection," in *Proceedings of the 26th ACM SIGKDD international conference on knowledge discovery & data mining*, 2020. doi: 10.1145/3394486.3403361.

[3] S. X. Rao, S. Zhang, Z. Han, Z. Zhang, W. Min, *et al.*, "xFraud: Explainable fraud transaction detection," *Proceedings of the VLDB Endowment*, 2021, doi: 10.14778/3494124.3494128.

[4] S. Krutikov, B. Khaertdinov, R. Kiriukhin, S. Agrawal, M. Ariannezhad, *et al.*, "Challenging gradient boosted decision trees with tabular transformers for fraud detection at booking.com." 2024. Available: https://arxiv.org/abs/2405.13692

[5] Y. Gorishniy, I. Rubachev, V. Khrulkov, and A. Babenko, "Revisiting deep learning models for tabular data," in *Advances in neural information processing systems*, 2021. Available: https://arxiv.org/abs/2106.11959

[6] N. Hollmann, S. Müller, L. Purucker, A. Krishnakumar, M. Körfer, *et al.*, "Accurate predictions on small data with a tabular foundation model," *Nature*, 2025, doi: 10.1038/s41586-024-08328-6.

[7] M. Weber, G. Domeniconi, J. Chen, D. K. I. Weidele, C. Bellei, *et al.*, "Anti-money laundering in bitcoin: Experimenting with graph convolutional networks for financial forensics." 2019. Available: https://arxiv.org/abs/1908.02591

[8] J. Tang, F. Hua, Z. Gao, P. Zhao, and J. Li, "GADBench: Revisiting and benchmarking supervised graph anomaly detection," in *Advances in neural information processing systems (datasets and benchmarks track)*, 2023. Available: https://arxiv.org/abs/2306.12251

[9] J. Gama, I. Žliobaitė, A. Bifet, M. Pechenizkiy, and A. Bouchachia, "A survey on concept drift adaptation," *ACM Computing Surveys*, 2014, doi: 10.1145/2523813.

[10] A. Dal Pozzolo, O. Caelen, Y.-A. Le Borgne, S. Waterschoot, and G. Bontempi, "Learned lessons in credit card fraud detection from a practitioner perspective," *Expert Systems with Applications*, 2014, doi: 10.1016/j.eswa.2014.02.026.

[11] A. Correa Bahnsen, D. Aouada, A. Stojanović, and B. Ottersten, "Feature engineering strategies for credit card fraud detection," *Expert Systems with Applications*, 2016, doi: 10.1016/j.eswa.2015.12.030.

[12] A. Dal Pozzolo, O. Caelen, R. A. Johnson, and G. Bontempi, "Calibrating probability with undersampling for unbalanced classification," in *IEEE symposium series on computational intelligence (SSCI)*, 2015. doi: 10.1109/SSCI.2015.33.

[13] C. Elkan, "The foundations of cost-sensitive learning," in *Proceedings of the 17th international joint conference on artificial intelligence (IJCAI)*, 2001.

[14] A. Correa Bahnsen, D. Aouada, and B. Ottersten, "Example-dependent cost-sensitive decision trees," *Expert Systems with Applications*, 2015, doi: 10.1016/j.eswa.2015.04.042.

[15] L. Grinsztajn, E. Oyallon, and G. Varoquaux, "Why do tree-based models still outperform deep learning on typical tabular data?" in *Advances in neural information processing systems (datasets and benchmarks track)*, 2022. Available: https://arxiv.org/abs/2207.08815

[16] R. Shwartz-Ziv and A. Armon, "Tabular data: Deep learning is not all you need," *Information Fusion*, 2022, doi: 10.1016/j.inffus.2021.11.011.

[17] F. T. Liu, K. M. Ting, and Z.-H. Zhou, "Isolation forest," in *Eighth IEEE international conference on data mining (ICDM)*, 2008. doi: 10.1109/ICDM.2008.17.

[18] B. Schölkopf, J. C. Platt, J. Shawe-Taylor, A. J. Smola, and R. C. Williamson, "Estimating the support of a high-dimensional distribution," *Neural Computation*, 2001, doi: 10.1162/089976601750264965.

[19] G. Pang, C. Shen, L. Cao, and A. van den Hengel, "Deep learning for anomaly detection: A review," *ACM Computing Surveys*, 2021, doi: 10.1145/3439950.

[20] S. Han, X. Hu, H. Huang, M. Jiang, and Y. Zhao, "ADBench: Anomaly detection benchmark," in *Advances in neural information processing systems (datasets and benchmarks track)*, 2022. Available: https://arxiv.org/abs/2206.09426

[21] F. Carcillo, Y.-A. Le Borgne, O. Caelen, Y. Kessaci, F. Oblé, *et al.*, "Combining unsupervised and supervised learning in credit card fraud detection," *Information Sciences*, 2021, doi: 10.1016/j.ins.2019.05.042.

[22] J. Jurgovsky, M. Granitzer, K. Ziegler, S. Calabretto, P.-É. Portier, *et al.*, "Sequence classification for credit-card fraud detection," *Expert Systems with Applications*, 2018, doi: 10.1016/j.eswa.2018.01.037.

[23] I. Padhi, Y. Schiff, I. Melnyk, M. Rigotti, Y. Mroueh, *et al.*, "Tabular transformers for modeling multivariate time series," in *IEEE international conference on acoustics, speech and signal processing (ICASSP)*, 2021. doi: 10.1109/ICASSP39728.2021.9414142.

[24] G. Aminian, A. J. Elliott, T. Li, T. H. Wong, V. C. Dehon, *et al.*, "FraudTransformer: Time-aware GPT for transaction fraud detection." 2025. Available: https://arxiv.org/abs/2509.23712

[25] Y. Dou, Z. Liu, L. Sun, Y. Deng, H. Peng, *et al.*, "Enhancing graph neural network-based fraud detectors against camouflaged fraudsters," in *Proceedings of the 29th ACM international conference on information & knowledge management (CIKM)*, 2020. doi: 10.1145/3340531.3411903.

[26] Y. Liu, X. Ao, Z. Qin, J. Chi, J. Feng, *et al.*, "Pick and choose: A GNN-based imbalanced learning approach for fraud detection," in *Proceedings of the web conference (WWW)*, 2021. doi: 10.1145/3442381.3449989.

[27] M. Lu, Z. Han, S. X. Rao, Z. Zhang, Y. Zhao, *et al.*, "BRIGHT – graph neural networks in real-time fraud detection," in *Proceedings of the 31st ACM international conference on information & knowledge management (CIKM)*, 2022. doi: 10.1145/3511808.3557136.

[28] G. Somepalli, M. Goldblum, A. Schwarzschild, C. B. Bruss, and T. Goldstein, "SAINT: Improved neural networks for tabular data via row attention and contrastive pre-training." 2021. Available: https://arxiv.org/abs/2106.01342

[29] N. Hollmann, S. Müller, K. Eggensperger, and F. Hutter, "TabPFN: A transformer that solves small tabular classification problems in a second," in *International conference on learning representations (ICLR)*, 2023. Available: https://arxiv.org/abs/2207.01848

[30] N. Hollmann, S. Müller, and F. Hutter, "Large language models for automated data science: Introducing CAAFE for context-aware automated feature engineering," in *Advances in neural information processing systems*, 2023. Available: https://arxiv.org/abs/2305.03403

[31] S. Hegselmann, A. Buendia, H. Lang, M. Agrawal, X. Jiang, *et al.*, "TabLLM: Few-shot classification of tabular data with large language models," in *International conference on artificial intelligence and statistics (AISTATS)*, 2023. Available: https://arxiv.org/abs/2210.10723

[32] X. Tan, Y. Ma, and X. Zhang, "Understanding structured financial data with LLMs: A case study on fraud detection," in *Proceedings of the annual meeting of the association for computational linguistics (ACL)*, 2026. doi: 10.18653/v1/2026.acl-long.1071.

[33] E. A. Lopez-Rojas, A. Elmir, and S. Axelsson, "PaySim: A financial mobile money simulator for fraud detection," in *28th european modeling and simulation symposium (EMSS)*, 2016.

[34] E. Altman, J. Blanuša, L. von Niederhäusern, B. Egressy, A. Anghel, *et al.*, "Realistic synthetic financial transactions for anti-money laundering models," in *Advances in neural information processing systems (datasets and benchmarks track)*, 2023. Available: https://arxiv.org/abs/2306.16424

[35] S. Jesus, J. Pombal, D. Alves, A. Cruz, P. Saleiro, *et al.*, "Turning the tables: Biased, imbalanced, dynamic tabular datasets for ML evaluation," in *Advances in neural information processing systems (datasets and benchmarks track)*, 2022. Available: https://arxiv.org/abs/2211.13358

[36] Y.-A. Le Borgne, W. Siblini, B. Lebichot, and G. Bontempi, *Reproducible machine learning for credit card fraud detection – practical handbook*. Université Libre de Bruxelles, 2022. Available: https://fraud-detection-handbook.github.io/fraud-detection-handbook/

[37] L. Xu, M. Skoularidou, A. Cuesta-Infante, and K. Veeramachaneni, "Modeling tabular data using conditional GAN," in *Advances in neural information processing systems*, 2019. Available: https://arxiv.org/abs/1907.00503